

Trabajo Práctico 2 — Intro a los Sistemas Distribuidos

SDN
2C - 2025

DOMINGUEZ LUCÍA, Juan Pablo	111401	jdominguezl@fi.ub.ar
ROGICA, Nicolas	94107	nrogica@fi.uba.ar
GAZCON, Felipe	110933	fgazcon@fi.uba.ar
PAGURA, Sebastian Martin	102649	spagura@fi.uba.ar
PALAZON, Martin	102679	mapalazon@fi.uba.ar

Índice

1. Introduccion	2
1.1. Suposiciones	2
2. Implementación	2
2.1. Topología	2
2.2. Firewall	2
2.3. Scripts	3
3. Uso de las Herramientas	3
4. Pruebas	4
4.1. Carga Firewall	4
4.2. Preparacion Mininet	4
4.3. Conexiones	4
4.4. Servidor iperf	5
4.5. Conexion exitosa	5
4.6. Conexion puerto 80	6
4.7. Segunda Regla	7
4.7.1. Conexion TCP	7
4.7.2. Conexion UDP	7
4.8. Tercer Regla	8
5. Preguntas a Responder	10
5.1. ¿cual es la diferencia entre un Switch y un router? ¿Que tienen en comun?	10
5.2. ¿Cual es la diferencia entre un Switch convencional y un Switch OpenFlow?	10
5.3. Se pueden reemplazar todos los routers de la Internet por Switches OpenFlow? Piense en el escenario interASes para elaborar su respuesta	10
6. Dificultades encontradas	11
7. Conclusión	11
8. README	12
8.1. tp2-redes	12
8.2. Repositorio git	12
8.3. INDICE	12
8.4. Como correr los scripts	13
8.4.1. Firewall	13
8.4.2. Explicacion de cada script:	13
8.4.3. Como ejecutarlo	13
8.4.4. Objetivo de cada script	14
8.5. EN CASO DE QUE LOS SCRIPTS NO FUNCIONEN	14

1. Introduccion

El presente informe reúne la documentación del trabajo practico 2 de la materia Sistemas Distribuidos (Redes), con la temática de Software Defined Networks (SDN) y Openflow. Para la elaboración de este se uso el lenguaje Python, la herramienta Mininet, OpenFlow, y el controlador POX. El enfoque del trabajo fueron las redes definidas por software (SDN) y el protocolo Openflow para manejarlas. La propuesta del mismo fue construir una topología dinámica e implementar un firewall en esta con el uso de Openflow.

1.1. Suposiciones

1. Para clarificar el armado de la topología, supusimos que solo hay un tipo de topología posible y es la de los switches en linea con dos hosts por extremo. Esto es mas una aclaración que suposición, puesto que el enunciado nos dejo con la duda.
2. Acorde a uno de los requerimientos que figuraban en la seccion Entrega, decidimos añadir el README.md al informe, aunque preferimos que usen el de Github.

2. Implementación

2.1. Topología

Siguiendo el orden propuesto en el enunciado del TP, desarrollaremos primero el funcionamiento del código encargado de el armado de nuestra topología, *topologia.py*. El código se inicia parseando los parámetros obtenidos de la terminal. El argumento esperado (y obligatorio) es el numero de switches que queremos en la red. Luego usando los comandos de mininet, ponemos 4 hosts (H1, H2, H3 y H4), hace una cadena de switches del largo indicado por comando y luego une dos hosts a cada extremo de la cadena.

2.2. Firewall

Para la implementación del Firewall se utilizo POX, como recomendó la catedra.

El firewall se inicializa dándole al gestor de la red la posibilidad de especificar en que switch quiere el Firewall (pueden ser varios). Esto lo hace con el parámetro *dpids* de la función *launch()*. El caso de que no se indique ninguno en especifico, se activara el firewall en todos los switches. Se hace uso de la función *load_rules_from_files()* (explicada mas adelante) para cargar las reglas a las que adhiere el firewall en *core.firewall_rules*, esto es, el núcleo del controlador POX. Se define *start_switch()*. Dependiendo de si el id del switch esta entre los especificados en el parámetro mencionado previamente activa el firewall en ese switch. Ademas, le instala las reglas usando la función *install_rules_on_connection()*. Por ultimo usa un comando que se asegura que la función *start_switch()* se ejecute automáticamente cada vez que un switch se conecte al controlador.

La previamente mencionada *load_rules_from_file()* es la encargada de cargar las reglas desde el json que las almacena. Se usa solo una vez al iniciar el controlador.

La función *install_rules_on_connection(connection, rules)* es quizá la mas importante del firewall. Es responsable de instalar en el switch *connection* flows de drop por cada regla. Por cada regla declarada en el *.json*, va a intentar extraer los datos de: acción de la regla, ip origen, puerto origen, ip destino, puerto destino, protocolo y prioridad de la regla. Luego invoca la función *format_rule()* a la que le pasa todos los datos recién recolectados, para que esta lo transforme en un string. Esto es con el único propósito de imprimir un log, no tendrá usos e impacto en el Firewall. En caso de que la acción sea distinta a "deny", se pasara por alto esta regla.

Luego se construye un objeto match, perteneciente a OpenFlow POX, y a continuación se van a agregar los atributos de la regla para completarlo. Este representa las condiciones que debe cumplir un paquete pasando por el FireWall para que la regla en cuestión lo afecte. Primero, traduce la

regla del protocolo de texto a un numero esperado por OpenFlow, por ejemplo: "*tcp*" - > 6, y la añade al objeto *match*. Esto solo soportara protocolos *tcp*, *udp* y *icmp*. Otro protocolo resultara en saltarse la regla. A continuación se agregan las partes de puertos e IPs. Con todos los datos registrados en el objeto, se hace un mensaje para enviar al switch, el cual va a contener el *match* configurado, la prioridad de la regla y una lista de acciones vacía, que indica que la accion sera un *drop*. Ya listo el mensaje, se envía al switch.

2.3. Scripts

Nuestra entrega también cuenta con los scripts necesarios para la ejecución de las pruebas del correcto funcionamiento del sistema. Contamos con los scripts de *firewall*, *h1_to_h4*, *h1_to_port_5001*, *h4_to_h1*, *normal_case*, *pingall* y *udp_to_80*. Ademas existe el archivo de configuración para los scripts.

3. Uso de las Herramientas

Las herramientas utilizadas en este Trabajo Practico son Mininet, POX, Wireshark e Iperf. Las primeras dos son directamente parte del codigo, mientras que las otras tienen un uso mas practico a la hora de medir y certificar el funcionamiento del sistema.

Usamos Mininet para simular nuestra SDN. Esto conlleva a lo tengamos que incorporar a nuestro codigo para especificar nuestra topologia, mas especificamente en *topologia.py*. Para el armado de la red, se utilizan los comandos mencionados en el enunciado del TP: *addHost*, para añadir un end user, *addSwitch*, para incorporar un switch a la red, y *addLink*, que enlaza un par de elementos de la topologia. Se puede intuitivamente pensar en el uso de estos comando de mininet como el armado de un grafo en otra materia.

POX es el controlador propuesto por la catedra. Dado que usamos OpenFlow para implementar el FireWall, es natural que POX este embebido en el codigo del FireWall. Mas especificamente vamos a usar los recursos de POX cada vez que queramos interactuar con los switches y sus reglas. Toda accion significativa en *firewall.py* tiene de por medio un metodo o recurso de POX, pero los mas importantes son *core.firewall_rules*, que registra las reglas de manera global, y *of.ofp_match()* y *of.ofp_flow_mod()* que manejan los objetos *match* y los flujos.

Wireshark e Iperf seran usados en la seccion de pruebas y demostraciones.

4. Pruebas

En esta seccion, se mostraran pruebas del funcionamiento de la topologia, bajo diversos escenarios clave. Disclaimer; las capturas de wireshark son en su mayoria dificiles de ver, pedimos de ser posible que traten de hacer zoom dado que toda la informacion en ellas es relevante por lo que no se podian reducir.

4.1. Carga Firewall

Se inicia POX con el comando mostrado por pantalla y se cargan las reglas.

```
vagrant@sisop:~/pox$ python3 ./pox.py log.level forwarding.l2_learning firewall
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:firewall: Iniciando firewall custom en TODOS los switches
INFO:firewall:Se cargaron 7 reglas desde /home/vagrant/pox/ext/reglas_fw.json
WARNING:version:POX requires one of the following versions of Python: 3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported version.
INFO:core:POX 0.7.0 (gar) is up.
```

Figura 1: Log de carga de reglas en Firewall.

4.2. Preparacion Mininet

Al iniciar Mininet, este construye la topología definida en topologia.py. En este caso, la herramienta genera dinámicamente cinco switches, tal como se especifica en el archivo. Una vez levantada la red virtual, Mininet permite visualizar fácilmente: la cantidad total de hosts, la cantidad total de switches, y todos los enlaces creados entre ellos.

```
vagrant@sisop:~/Desktop/Facultad/Redes/TP2/tp2-redes$ sudo mn --custom topologia.py --topo customTopo,switches=5 --controller remote
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3 s4 s5
*** Adding links:
(h1, s1) (h2, s1) (h3, s5) (h4, s5) (s1, s2) (s2, s3) (s3, s4) (s4, s5)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 5 switches
s1 s2 s3 s4 s5 ...
*** Starting CLI:
mininet>
```

Figura 2: Log de la preparacion de la topologia en Mininet.

4.3. Conexiones

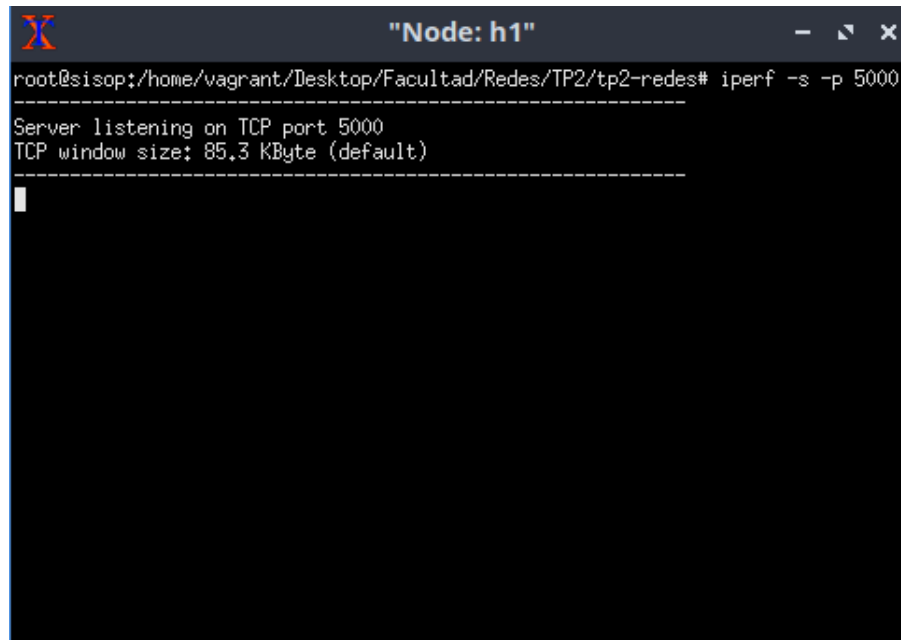
Aquí puede observarse a qué interfaz quedó conectado cada host y cada switch, lo que permite identificar claramente los enlaces creados dentro de la topología virtual.

```
h1-eth0<->s1-eth2 (OK OK)
h2-eth0<->s1-eth3 (OK OK)
h3-eth0<->s5-eth2 (OK OK)
h4-eth0<->s5-eth3 (OK OK)
s1-eth1<->s2-eth1 (OK OK)
s2-eth2<->s3-eth1 (OK OK)
s3-eth2<->s4-eth1 (OK OK)
s4-eth2<->s5-eth1 (OK OK)
```

Figura 3: Log de conexiones e interfaces.

4.4. Servidor iperf

Se inicia un servidor iperf en el host h1, configurado para utilizar el protocolo TCP y escuchar conexiones entrantes en el puerto 5000.

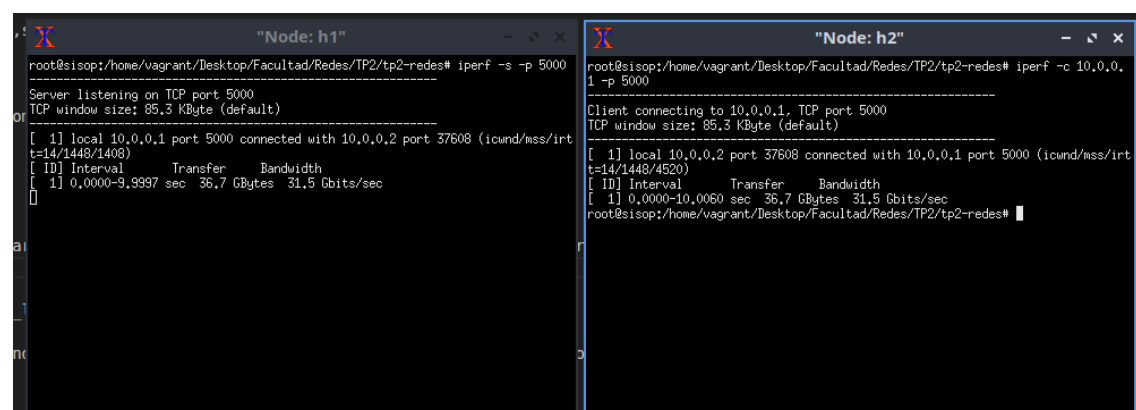


```
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -s -p 5000
-----
Server listening on TCP port 5000
TCP window size: 85.3 KByte (default)
-----
```

Figura 4: Terminal inicio de servidor iperf.

4.5. Conexion exitosa

En la captura se observa que la conexión entre ambos hosts se estableció correctamente. Además, en Wireshark puede verse el intercambio de paquetes correspondiente a la comunicación TCP, lo que confirma que el tráfico fluye sin restricciones en este escenario.



```
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -s -p 5000
-----
Server listening on TCP port 5000
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 5000 connected with 10.0.0.2 port 37608 (icwnd/mss/irt
t=14/1448/1408)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-9.9997 sec 36.7 GBytes 31.5 Gbits/sec

root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes#

root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -c 10.0.0.1 -p 5000
-----
Client connecting to 10.0.0.1, TCP port 5000
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 37608 connected with 10.0.0.1 port 5000 (icwnd/mss/irt
t=14/1448/4520)
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0060 sec 36.7 GBytes 31.5 Gbits/sec

root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes#
```

Figura 5: Logs de conexion de ambos lados.

La siguiente captura fue tomada desde el host h1. Se aprecia el SYN, EL SYN ACK y el correspondiente intercambio entre ambos hosts

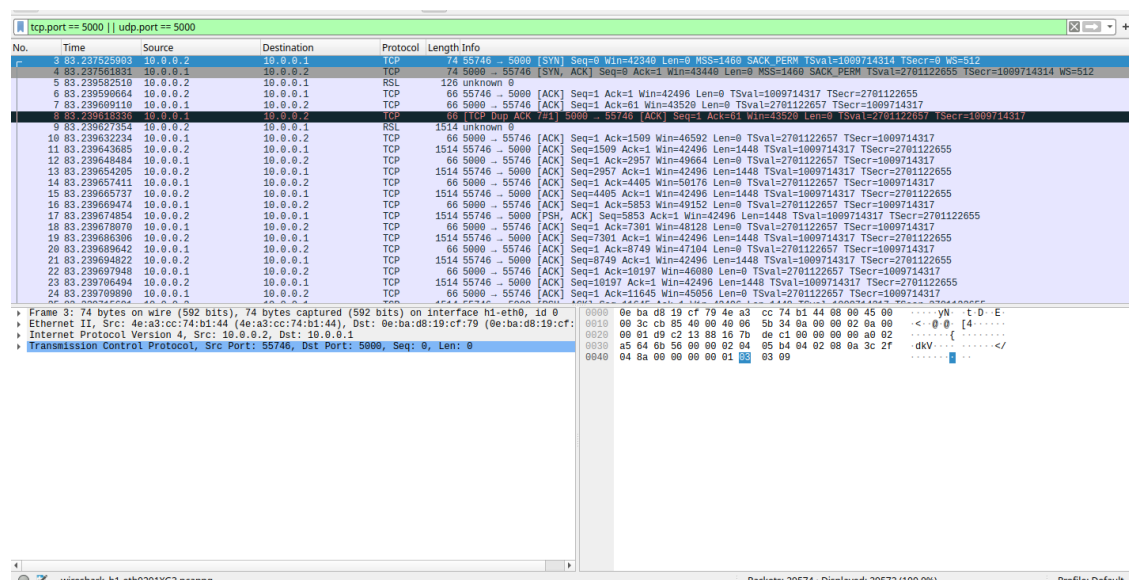


Figura 6: Wireshark de la conexion.

4.6. Conexion puerto 80

Ahora, si repetimos la misma prueba pero configuramos h1 para escuchar con iperf en el puerto 80, el firewall debería bloquear este tráfico. Como consecuencia, h2 no debería recibir ninguna estadística de rendimiento, ya que la conexión TCP no logrará establecerse y no habrá transferencia de datos.

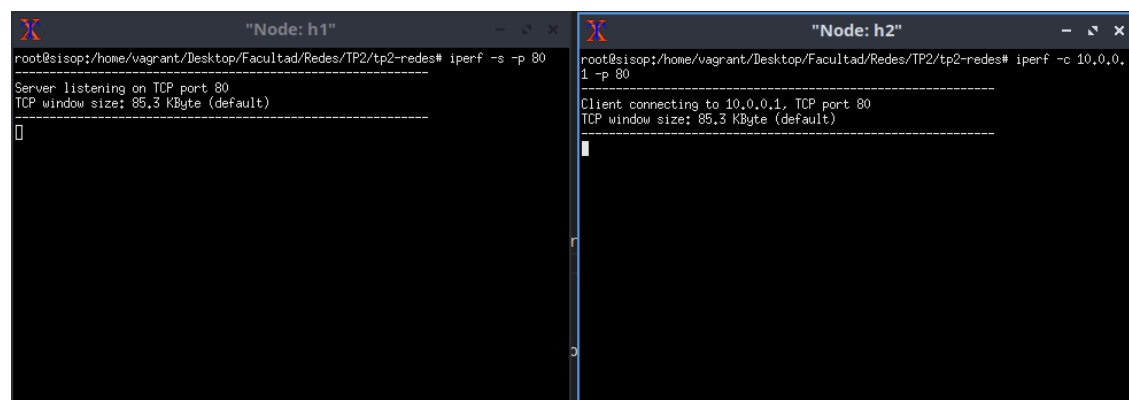


Figura 7: Logs de conexion inexitosa de ambos lados.

En la siguiente captura de Wireshark puede observarse cómo h2 intenta iniciar la conexión enviando múltiples segmentos SYN hacia h1. Si bien estos paquetes alcanzan al switch, el firewall los bloquea, impidiendo que el servidor responda y evitando que se establezca la conexión TCP.

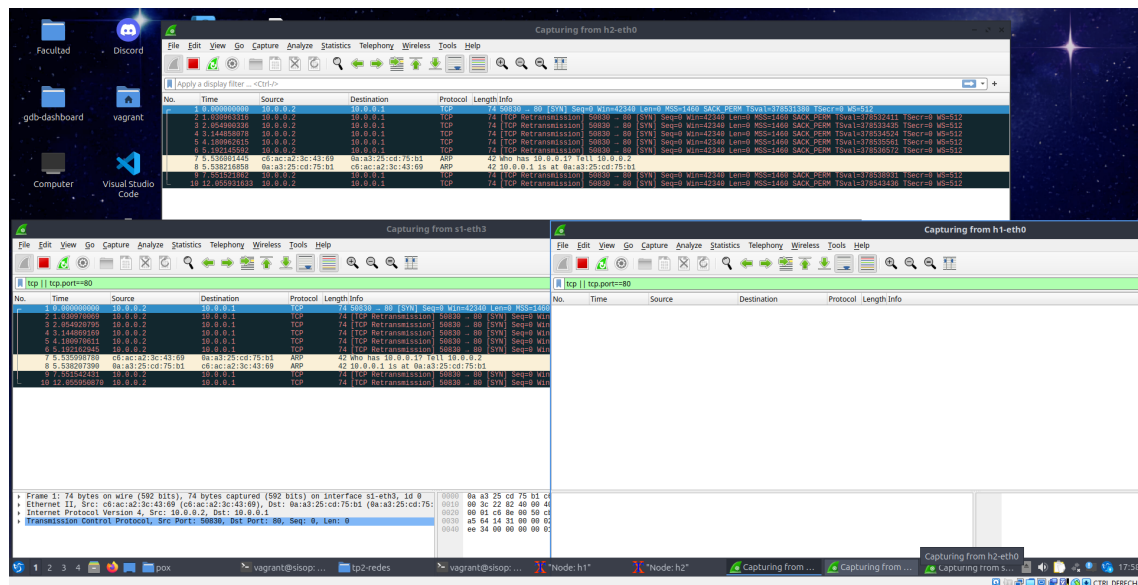


Figura 8: Wireshark de la conexion.

4.7. Segunda Regla

Este comportamiento es más específico: los paquetes capturados corresponden a tráfico proveniente de h1, enviado desde el puerto 5001 utilizando el protocolo UDP.

4.7.1. Conexion TCP

Primero, si la conexión fuera TCP, por ejemplo, el funcionamiento sería correcto y la comunicación se establecería sin problemas.

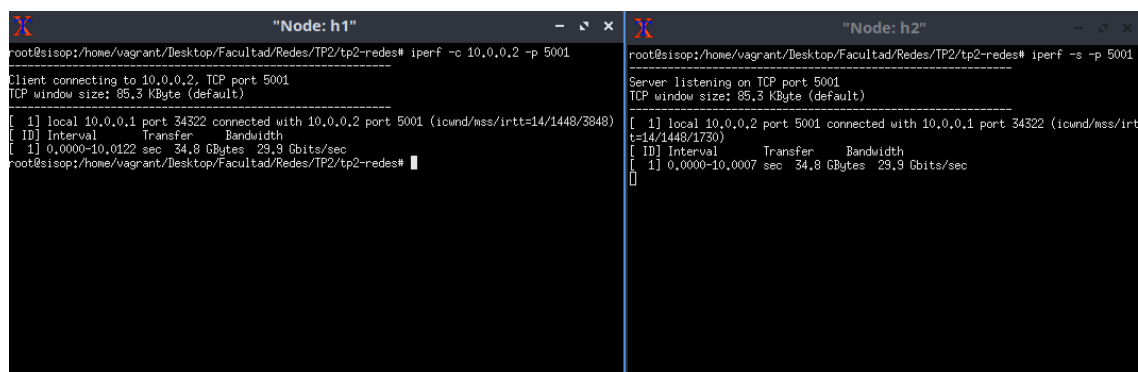


Figura 9: Logs de conexion TCP.

4.7.2. Conexion UDP

Ahora en el caso pedido:


```

"Node: h1"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -c 10.0.0.2 -p 5001
Client connecting to 10.0.0.2, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kaiman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.1 port 46099 connected with 10.0.0.2 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-0.0153 sec  1.25 MBytes  1.05 Mbits/sec
[ 1] Sent 896 datagrams
[ 5] WARNING: did not receive ack of last datagram after 10 tries.
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes#

"Node: h2"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -s -p 5001
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)

[]

```

Figura 10: Logs de conexion UDP.

Como se observa en la captura, h2 intenta establecer la conexión, pero nunca recibe los ACK correspondientes desde h1 debido al bloqueo del firewall. Al no obtener respuesta luego de varios intentos de retransmisión, iperf genera un warning, indicando que no fue posible completar el handshake TCP.

Como se puede apreciar en wireshark, a h2 nunca le llega nada

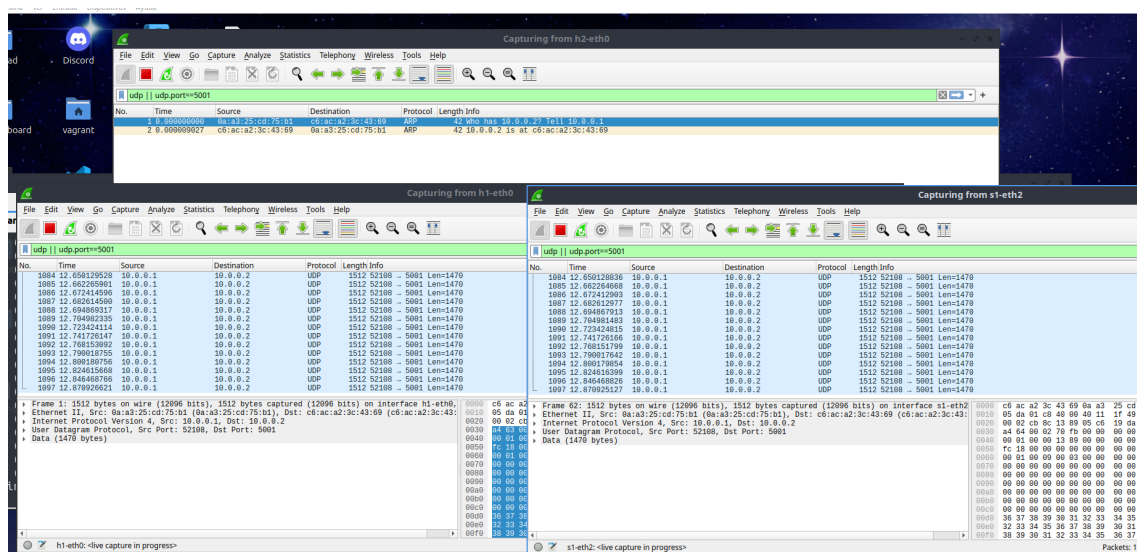
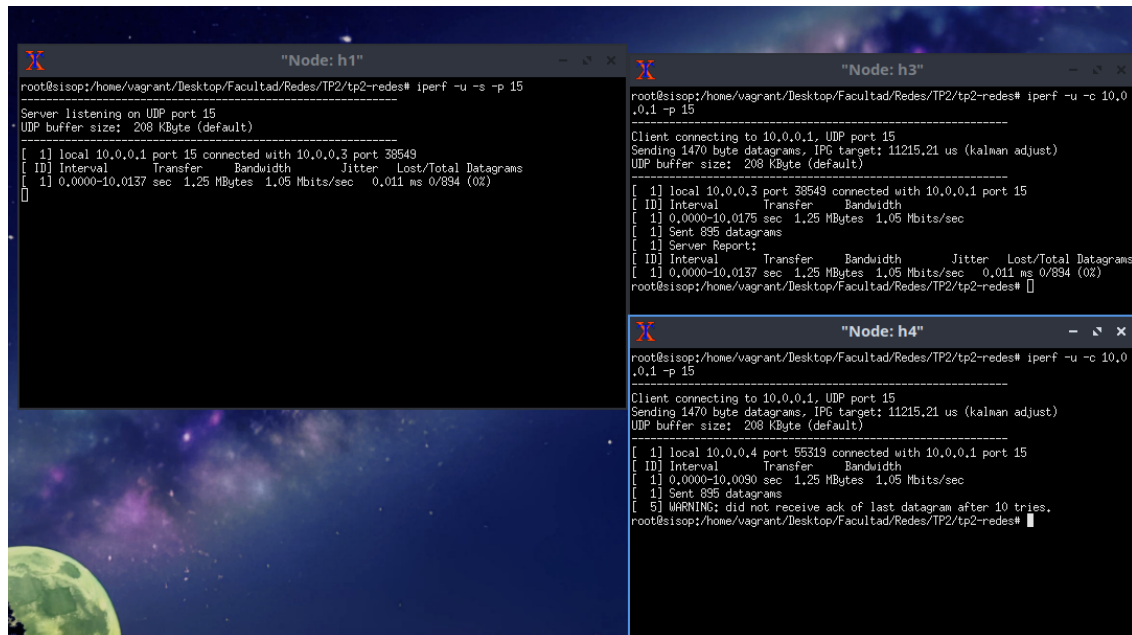


Figura 11: Wireshark de conexion.

4.8. Tercer Regla

Se configuró el firewall para que h1 y h4 no puedan comunicarse entre sí, independientemente del puerto o del protocolo utilizado. Es decir, cualquier intento de conexión entre estos dos hosts será bloqueado sin excepción.

En esta captura puede observarse que, utilizando el mismo protocolo y el mismo puerto, el host h3 logra conectarse correctamente con h1, mientras que h4 no puede establecer la comunicación. Esto evidencia que el firewall está aplicando la regla de bloqueo específicamente entre h1 y h4, sin afectar a otros hosts que utilizan las mismas condiciones de conexión.



```

"Node: h1"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -s -p 15
Server listening on UDP port 15
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.1 port 15 connected with 10.0.0.3 port 38549
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0137 sec 1.25 MBytes 1.05 Mbits/sec 0.011 ms 0/894 (0%)

"Node: h3"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -c 10.0.0.1 -p 15
Client connecting to 10.0.0.1, UDP port 15
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.3 port 38549 connected with 10.0.0.1 port 15
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0175 sec 1.25 MBytes 1.05 Mbits/sec
[ 1] Sent 895 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0137 sec 1.25 MBytes 1.05 Mbits/sec 0.011 ms 0/894 (0%)

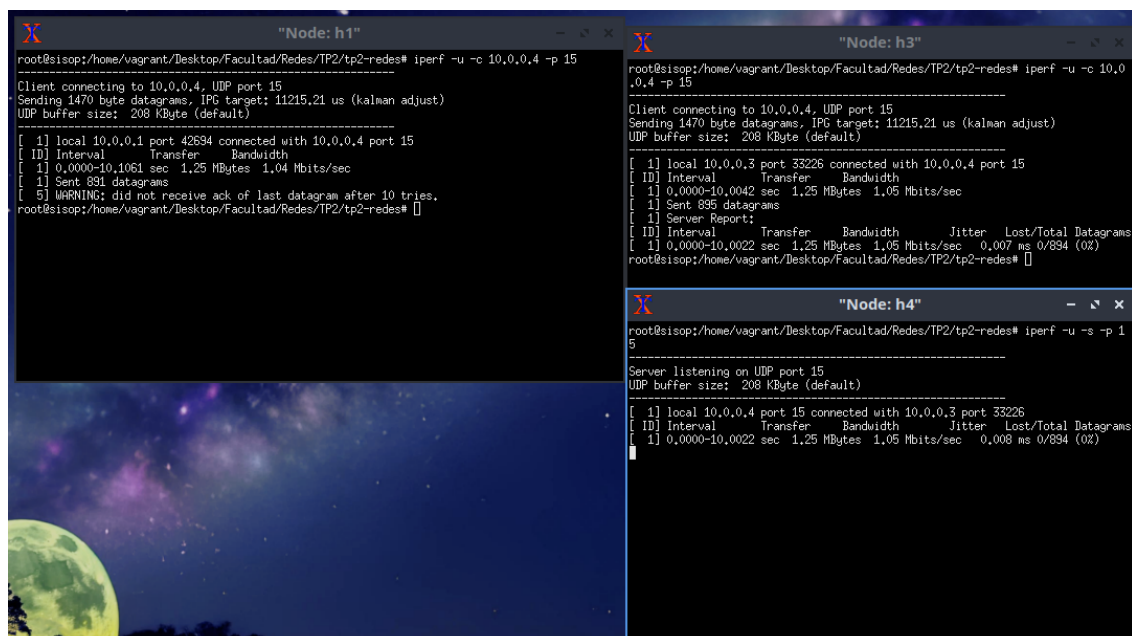
"Node: h4"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -c 10.0.0.1 -p 15
Client connecting to 10.0.0.1, UDP port 15
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.4 port 55319 connected with 10.0.0.1 port 15
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0090 sec 1.25 MBytes 1.05 Mbits/sec
[ 1] Sent 895 datagrams
[ 5] WARNING: did not receive ack of last datagram after 10 tries.

```

Figura 12: Intento de conexión H1-H4.

Lo mismo ocurre si el servidor se ejecuta en h4: los hosts no bloqueados pueden conectarse sin inconvenientes, mientras que h1 continúa impedido de establecer comunicación, tal como lo especifica la política del firewall.



```

"Node: h1"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -c 10.0.0.4 -p 15
Client connecting to 10.0.0.4, UDP port 15
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.1 port 42694 connected with 10.0.0.4 port 15
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.1061 sec 1.25 MBytes 1.04 Mbits/sec
[ 1] Sent 891 datagrams
[ 5] WARNING: did not receive ack of last datagram after 10 tries.

"Node: h3"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -c 10.0.0.4 -p 15
Client connecting to 10.0.0.4, UDP port 15
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.3 port 33226 connected with 10.0.0.4 port 15
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0042 sec 1.25 MBytes 1.05 Mbits/sec
[ 1] Sent 895 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0022 sec 1.25 MBytes 1.05 Mbits/sec 0.007 ms 0/894 (0%)

"Node: h4"
root@sisop:/home/vagrant/Desktop/Facultad/Redes/TP2/tp2-redes# iperf -u -s -p 15
Server listening on UDP port 15
UDP buffer size: 208 KByte (default)

[ 1] local 10.0.0.4 port 15 connected with 10.0.0.3 port 33226
[ ID] Interval      Transfer    Bandwidth    Jitter    Lost/Totals Datagrams
[ 1] 0.0000-10.0022 sec 1.25 MBytes 1.05 Mbits/sec 0.008 ms 0/894 (0%)

```

Figura 13: Intento de conexión H4-H1.

5. Preguntas a Responder

5.1. ¿cual es la diferencia entre un Switch y un router? ¿Que tienen en comun?

Un Router es un dispositivo de la capa de red (L3) que interconecta diferentes redes entre si mientras que un switch es un dispositivo de la capa de enlace (l3) que interconecta hosts dentro de una misma red local (LAN).

Switch	Router
-Utiliza direcciones MAC	-Utiliza direcciones IP
-Funciona en la capa de enlace (L2)	-Funciona en la capa de red (L3)
-Conecta hosts dentro de una misma red local	-Conecta distintas redes o subredes
-Aprende qué MAC está en qué puerto	-Tiene una tabla de ruteo, aplica LPM y decide a qué red mandar cada paquete
-No hace routing	-Puede fragmentar paquetes si el MTU lo requiere

Cuadro 1: Cuadro comparativo entre Switch y Router

¿Qué tienen en común? Ambas son esenciales para el correcto funcionamiento de una red completa (lan interna + conexión entre redes). Ambos interconectan dispositivos. Manejan tablas internas (tablas de MAC, y tablas de ruteo), poseen puertos, manejan buffers, descartan paquetes si están llenos.

5.2. ¿Cual es la diferencia entre un Switch convencional y un Switch OpenFlow?

Un switch convencional incorpora en el propio equipo tanto el plano de datos como el plano de control. Toda la lógica ocurre dentro del mismo dispositivo. En cambio, un switch OpenFlow separa estos planos. El plano de datos permanece en el switch, pero el plano de control se externaliza a un controlador SDN. La comunicación entre el controlador y el switch se realiza mediante el protocolo OpenFlow, que permite controlar las reglas en la tabla de flujos del switch. El controlador puede operar bajo dos dinámicas: Configuración inicial: el controlador asigna funciones una única vez cuando se instala el switch. Configuración dinámica: el controlador define constantemente nuevas políticas.

5.3. Se pueden reemplazar todos los routers de la Internet por Switches OpenFlow? Piense en el escenario interASes para elaborar su respuesta

No es posible reemplazar todos los routers de internet por switches OpenFlow. En el escenario de comunicación entre sistemas autónomos, los routers ejecutan el Border Gateway Protocol, el cual necesita tablas grandes de ruteo, toma de decisiones distribuidas (sin un controlador externo que provea la información) y además tener en cuenta políticas comerciales que no pueden ser controladas por una única entidad. Estos factores hacen que los switches de OpenFlow, aunque se puedan usar para construir routers SDN avanzados, no encajen con la arquitectura distribuida del internet global.

6. Dificultades encontradas

Para algunos de los integrantes del grupo, el uso de POX en Ubuntu se vio afectado por las versiones de Python y sistemas operativos que utilizaba cada uno. Si bien la documentación de las herramientas necesitadas para el TP resultaron mas que suficientes, la dificultad del TP en si residía en el aprendizaje del uso de estas. Trabajando en conjunto ambas condiciones se volvieron triviales.

7. Conclusión

A lo largo del trabajo se logró comprender en profundidad el funcionamiento de una red definida por software (SDN) y el rol que ocupa el controlador en la administración de la red. La implementación de un firewall utilizando POX y OpenFlow permitió observar en la práctica la separación entre el plano de control y el plano de datos, y cómo esta separación habilita una red mucho más flexible, programable y centralizada.

El trabajo permitió también entender mejor el funcionamiento del aprendizaje de direcciones (L2 learning) y la importancia de protocolos como ARP para el establecimiento de la comunicación inicial entre hosts. Se comprobó que bloquear ciertos tipos de tráfico puede impactar en la capacidad del switch para aprender MACs y, en consecuencia, afectar el forwarding en toda la topología.

8. README

Añadimos tambien el README.md al informe, dado que resulto algo ambiguo un de los requerimientos de entrega: ".^{En} el informe debe figurar un README de cómo correr los diferentes scripts y qué se espera luego de la ejecución de cada uno.". En todo caso, decidimos integrarlo para estar seguros. Cabe recalcar que la integración al informe se hizo con librerías ofrecidas por Overleaf, por lo que el formato puede estar algo desalineado por lo que alentamos que utilicen directamente el readme en el repositorio de github.

8.1. tp2-redes

8.2. Repositorio git

<https://github.com/spagura/tp2-redes>

8.3. INDICE

- Como correr los scripts¹
 - Firewall²
 - Explicación de cada script³
 - normal_case.sh⁴
 - firewall.sh⁵
 - pingall.sh⁶
 - h1_to_h4.sh⁷
 - h4_to_h1.sh⁸
 - h1_to_port_5001.sh⁹
 - udp_to_80.sh¹⁰
 - Como ejecutarlo¹¹
 - Objetivo de cada script¹²
- EN CASO DE QUE LOS SCRIPTS NO FUNCIONEN¹³
 - POX¹⁴
 - Levantar POX con firewall custom¹⁵
 - Para crear el mininet con N switches (sin protocolo)¹⁶
 - Forma de correrlo¹⁷
 - IPERF¹⁸

```

1#como-correr-los-scripts
2#firewall
3#explicación-de-cada-script
4#normal_casesh
5#firewallsh
6#pingallsh
7#h1_to_h4sh
8#h4_to_h1sh
9#h1_to_port_5001sh
10#udp_to_80sh
11#como-ejecutarlo
12#objetivo-de-cada-script
13#en-caso-de-que-los-scripts-no-funcionen
14#pox
15#levantar-pox-con-firewall-custom
16#para-crear-el-mininet-con-n-switches-sin-protocolo
17#forma-de-correrlo
18#iperf

```

8.4. Como correr los scripts

Abre el archivo de configuración y actualiza las variables para que coincidan con tu sistema

Todos los comandos de los scripts deben realizarse en la carpeta scripts

8.4.1. Firewall

Antes de ejecutar cualquier script de Mininet, ejecutá el script del firewall. Debes Abrió una terminal en la carpeta scripts y escribí:

```
bash firewall.sh
```

8.4.2. Explicacion de cada script:

normal_case.sh Inicia la topología de manera normal.

Corre conexiones TCP entre los hosts que **no serían bloqueadas** por las reglas de firewall. Su objetivo es mostrar cómo funciona la red cuando todo está bien.

firewall.sh Inicia Mininet con el firewall de POX y carga tus reglas.

pingall.sh Ejecuta **pingall** dentro de la topología de Mininet.

Este script muestra rápidamente que el aprendizaje de nivel 2 está correctamente configurado y funcionando con POX.

h1_to_h4.sh Levanta un servidor TCP en h4. Desde h1 corre un **iperf** TCP hacia h4. Quedará intentando conectar, debés interrumpirlo con **Ctrl + C**.

También levanta un servidor UDP en h4 e intenta conectarse desde h1.

Luego muestra los logs de h4 para ambos servidores, donde **no debería aparecer ningún paquete recibido**, indicando que el firewall lo bloquea correctamente.

h4_to_h1.sh Lo mismo que el anterior, pero en dirección invertida.

h1_to_port_5001.sh Crea servidores UDP en H2 y H3. Intenta alcanzarlos desde H1 y luego muestra los logs de ambos servidores. Los logs deben estar vacíos, ya que los paquetes deben ser bloqueados por el firewall.

udp_to_80.sh Para probar la regla del firewall que bloquea **todo puerto 80**, levanta un servidor UDP en cada host y luego intenta alcanzarlos desde todos los demás hosts. Muestra los logs de los 4 servidores: todos deben estar vacíos.

8.4.3. Como ejecutarlo

Todos los scripts siguen el mismo patrón:

Iniciar Mininet + Firewall POX:

```
bash firewall.sh
```

Ejecutar pruebas de conectividad:

En otra terminal:

```
bash pingall.sh
```

Ejecutar pruebas TCP/UDP:

```
bash h1_to_h4.sh
```

```
bash h4_to_h1.sh
```

```
bash h1_to_port_5001.sh
```

```
bash udp_to_80.sh
```

Caso normal sin bloqueo del firewall:

```
bash normal_case.sh
```

8.4.4. Objetivo de cada script

normal_case.sh: comportamiento base.

firewall.sh: carga las reglas del firewall.

pingall.sh: verifica conectividad.

hX_to_hY scripts: testean tráfico en una dirección.

port/udp scripts: validan reglas de puerto/protocolo.

8.5. EN CASO DE QUE LOS SCRIPTS NO FUNCIONEN

POX Luego de clonar el repositorio de POX provisto por la cátedra (link incluido en el PDF del trabajo práctico), es importante ejecutar todos los comandos de POX dentro de la carpeta principal del proyecto, es decir, en el directorio donde se encuentra el archivo `pox.py`. Esto garantiza el correcto funcionamiento del controlador.

Para usar `pox` con la logica de learning switch usar el comando:

```
python pox.py forwarding.l2_learning
```

Si queremos especificar el nivel de logging usar:

```
python pox.py log.level --DEBUG forwarding.l2_learning (o INFO, WARNING, ERROR)
```

Por defecto `pox` escucha a todas las ips en el puerto 6633. Pero se puede especificar otra ip o puerto de la siguiente manera

```
python pox.py openflow.of_01 --port=<puerto> --address=<ip> forwarding.l2_learning
```

Levantar POX con firewall custom Para usar POX con el firewall programado para este trabajo practico, usar el comando:

```
python pox.py firewall forwarding.l2_learning
```

En el firewall custom se puede indicar con el parametro `-dpids` los switches a los que se les aplica el firewall, por ejemplo:

```
python pox.py firewall --dpids=1 forwarding.l2_learning
```

Aplica el firewall solo al switch 1, si no se indica el parametro se aplica a todos los switches de la topologia.

Para crear el mininet con N switches (sin protocolo) Nos ubicamos en la carpeta del trabajo práctico, donde se encuentra el archivo `topologia.py`.

```
sudo mn --custom topologia.py --topo customTopo,switches=N --controller=none --switch=ovsk
```

Si estan usando UBUNTU, dentro de mininet hay que mandar

```
mininet> sh ovs-vsctl set-fail-mode s1 standalone
mininet> sh ovs-vsctl set-fail-mode s2 standalone
mininet> sh ovs-vsctl set-fail-mode s3 standalone
mininet> sh ovs-vsctl set-fail-mode s4 standalone
mininet> sh ovs-vsctl set-fail-mode s5 standalone
....
```

Esto hace que si al switch no se le indica un controlador, se comporta como un switch learning normal (`l2_learning`)

Para levantar mininet con n switches y pox como controlador

```
sudo mn --custom topologia.py --topo customTopo,switches=N --controller=remote --switch=ovsk
```

Forma de correrlo Para ejecutar el trabajo de la forma esperada, es necesario abrir dos terminales: una para levantar POX con el firewall personalizado junto con `l2_learning`, y otra para iniciar Mininet utilizando POX como controlador.

IPERF Abrir terminales con xterm en el host que prefieran:

```
xterm h1
xterm h2
xterm h3
...
```

Montar servidor TCP

Dentro de xterm en el host donde desee levantar el servidor:

```
iperf -s -p "puerto a eleccion"
```

Montar servidor UDP

```
iperf -s -u -p "puerto a eleccion"
```

Conectarse a servidor TCP

Desde otro host, intente conectarse al servidor levantado con:

```
iperf -c "ip de host" -p "puerto"
```

Conectarse a servidor UDP

```
iperf -c "ip de host" -u -p "puerto"
```